



# GESTION DES PROCESSUS ET DES RESSOURCES PAR UN SYSTÈME D'EXPLOITATION

## Objectifs de cette leçon

- Comprendre ce qu'est un processus et comment l'OS le gère
- Connaître les 3 états d'un processus
- Comprendre l'ordonnancement (FIFO et Round-Robin)
- Maîtriser les notions de ressource partagée et d'exclusion mutuelle
- Identifier et expliquer un interblocage (deadlock)

 **À savoir :** Ces notions sont au programme de Terminale NSI et tombent régulièrement au Bac.

## Plan du cours

1. **Le système d'exploitation et les processus**
2. **Les états d'un processus**
3. **L'ordonnancement des processus**
4. **L'ordonnancement en temps partagé (Round-Robin)**
5. **La gestion des ressources et l'exclusion mutuelle**
6. **L'interblocage (Deadlock)**
7. **Synthèse et questions type Bac**

# 1. Le système d'exploitation et les processus

## ? Le problème de départ

Imagine que tu es devant ton ordinateur. Tu fais plusieurs choses en même temps :

-  Tu écoutes de la musique
-  Tu navigues sur Internet
-  Tu rédiges un document

**? Question :** Comment l'ordinateur fait-il pour exécuter plusieurs programmes en même temps ?

## Le processeur (CPU) : son rôle et sa limite

Le processeur (CPU = Central Processing Unit) est le cerveau de l'ordinateur. C'est lui qui exécute les instructions.

**⚠ Règle fondamentale :**  
**Un processeur ne peut exécuter qu'UNE SEULE instruction à la fois.**

 Donc : à un instant donné, un seul programme est réellement exécuté.

## Le rôle du système d'exploitation (OS)

Pour résoudre ce problème, tous les ordinateurs utilisent un logiciel spécial :

 **Définition :**

**Le système d'exploitation (OS = Operating System) est un programme qui :**

- gère l'exécution des autres programmes
- attribue le processeur à chaque programme
- organise l'utilisation de toutes les ressources (mémoire, disque, réseau...)

Exemples d'OS : Windows, Linux, Android, macOS, iOS

## **Notion de processus**

 **Définition à retenir :**  
**Un processus est un programme en cours d'exécution.**

 **Attention à la différence :**

| Programme                                                      | Processus                                         |
|----------------------------------------------------------------|---------------------------------------------------|
| Une suite d'instructions écrites et enregistrées sur le disque | Le programme au moment où il s'exécute en mémoire |
| Comme une recette de cuisine écrite sur un papier              | Comme la recette en train d'être cuisinée         |
| Inactif, statique                                              | Actif, dynamique — il consomme des ressources     |

**Exemples concrets :**

- Ouvrir Firefox → crée un processus Firefox
- Lancer un jeu vidéo → crée un processus pour ce jeu
- Ouvrir Word → crée un processus Word
- Plusieurs onglets ouverts dans Chrome → plusieurs processus Chrome

## **L'illusion de simultanéité**

Le système d'exploitation crée l'illusion que tout fonctionne en même temps. Voici comment :

| Étape                        | Ce qui se passe réellement                    |
|------------------------------|-----------------------------------------------|
| 1. P1 (musique) s'exécute    | Le CPU joue 0,001 seconde de musique          |
| 2. P2 (navigateur) s'exécute | Le CPU affiche une partie de la page web      |
| 3. P3 (Word) s'exécute       | Le CPU traite quelques caractères du document |
| 4. On revient à P1           | Le cycle recommence à toute vitesse           |

 **À retenir :**

**Les processus ne s'exécutent PAS vraiment en même temps.**

L'OS les fait alterner si rapidement (milliers de fois par seconde) que l'on a l'impression d'une exécution simultanée.

## 2. Les états d'un processus

Un processus ne fait pas toujours la même chose. À un instant donné, il peut être dans l'un des 3 états suivants :

|                                                                                                                                                                                                                            |                                                                                                                                                                                                                                                              |                                                                                                                                                                                                                                                                 |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  <b>PRÊT</b><br>Le processus est chargé en mémoire et attend son tour pour utiliser le CPU.<br>💡 Analogie : tu fais la queue à la caisse. |  <b>EN EXÉCUTION (ELU)</b><br>Le processus est en train d'utiliser le CPU. C'est lui qui travaille réellement.<br>💡 Analogie : tu es à la caisse et tu passes tes articles. |  <b>BLOQUÉ</b><br>Le processus attend une ressource (imprimante, fichier...). Il ne peut pas continuer.<br>💡 Analogie : tu attends que quelqu'un te rappelle pour continuer. |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

### Les transitions entre les états

Un processus change d'état en permanence. Voici les transitions possibles :

| Transition         | Cause                                  | Exemple concret                          |
|--------------------|----------------------------------------|------------------------------------------|
| PRÊT → EXÉCUTION   | L'ordonnanceur choisit ce processus    | Le CPU est libre, Firefox prend la main  |
| EXÉCUTION → PRÊT   | Le processus est interrompu (time-out) | Le CPU passe à un autre processus        |
| EXÉCUTION → BLOQUÉ | Le processus attend une ressource      | Le processus demande à imprimer          |
| BLOQUÉ → PRÊT      | La ressource est maintenant disponible | L'imprimante est libre, retour à la file |

### Exemple complet : imprimer un document

**Situation : tu travailles sur un PDF et tu cliques sur « Imprimer »**

| Étape | État du processus | Ce qui se passe |
|-------|-------------------|-----------------|
|-------|-------------------|-----------------|

|   |             |                                                                           |
|---|-------------|---------------------------------------------------------------------------|
| 1 | ● EXÉCUTION | Tu travailles sur le PDF. Le processus utilise le CPU.                    |
| 2 | ● BLOQUÉ    | Tu cliques sur Imprimer. L'imprimante est occupée par un autre processus. |
| 3 | ● BLOQUÉ    | Le processus attend. Il ne consomme plus le CPU.                          |
| 4 | ● PRÊT      | L'imprimante est libre ! Le processus rentre dans la file d'attente.      |
| 5 | ● EXÉCUTION | L'ordonnanceur lui donne le CPU. L'impression commence.                   |

### ✗ Erreurs fréquentes à éviter :

- « Bloqué = le programme a planté » → FAUX ! C'est un état normal, le processus attend juste.
- « Un processus bloqué utilise le CPU » → FAUX ! Il attend sans consommer le processeur.
- « Tous les processus s'exécutent en même temps » → FAUX ! C'est une illusion.

## 3. L'ordonnancement des processus

### Pourquoi ordonner les processus ?

Rappel : plusieurs processus peuvent être en état PRÊT en même temps, mais le CPU ne peut en traiter qu'un seul.

#### Problème :

Lequel des processus prêts doit utiliser le CPU en premier ? Et dans quel ordre ?

### L'ordonnanceur (scheduler)

#### Définition :

L'ordonnanceur est un composant du système d'exploitation qui décide quel processus va utiliser le CPU, et pendant combien de temps.

 Analogie : l'ordonnanceur est comme le chef de gare qui décide quel train (processus) peut avancer sur la voie (CPU).

### Algorithme FIFO (Premier arrivé = Premier servi)

**FIFO = First In, First Out.**

Principe simple : le processus qui est arrivé le premier dans la file des prêts est exécuté en premier.

**Exemple avec 3 processus :**

| Processus | Arrive à | Durée d'exécution | Ordre d'exécution           |
|-----------|----------|-------------------|-----------------------------|
| P1        | t = 0    | 4 secondes        | 1er → exécuté de t=0 à t=4  |
| P2        | t = 1    | 3 secondes        | 2ème → exécuté de t=4 à t=7 |

|    |         |            |                                 |
|----|---------|------------|---------------------------------|
| P3 | $t = 2$ | 2 secondes | 3ème → exécuté de $t=7$ à $t=9$ |
|----|---------|------------|---------------------------------|

 **Chronogramme FIFO :**

Temps : | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |

CPU : | P1 | P1 | P2 | P2 | P3 | P3 |

 Inconvénient de FIFO : si P1 est très long, P2 et P3 attendent longtemps, même si leur tâche est courte.

## 4. L'ordonnancement en temps partagé (Round-Robin)

### Le problème de FIFO et la solution

Avec FIFO, un processus peut monopoliser le CPU pendant très longtemps. Les autres sont bloqués en attente.

#### **Solution : le temps partagé (Round-Robin)**

Chaque processus reçoit une petite durée d'utilisation du CPU, appelée :

### **QUANTUM (ou tranche de temps)**

Quand son quantum est épuisé, le processus est mis en attente et un autre processus prend le CPU.

### Fonctionnement du Round-Robin

| Étape                | Description                                       |
|----------------------|---------------------------------------------------|
| 1. File d'attente    | Les processus prêts sont dans une file circulaire |
| 2. Premier processus | P1 reçoit le CPU pour un quantum (ex : 2ms)       |
| 3. Quantum épuisé    | P1 est interrompu et mis à la fin de la file      |
| 4. Tour de P2        | P2 reçoit le CPU pour un quantum                  |
| 5. Tour de P3        | P3 reçoit le CPU pour un quantum                  |
| 6. Retour à P1       | P1 reprend son tour → le cycle recommence         |

#### **Exemple concret (quantum = 2 unités) :**

P1 (durée 5u), P2 (durée 3u), P3 (durée 4u)

**Exécution :**

- $t=0$  à  $t=2$  : P1 s'exécute (quantum épuisé, reste 3u)
- $t=2$  à  $t=4$  : P2 s'exécute (quantum épuisé, reste 1u)
- $t=4$  à  $t=6$  : P3 s'exécute (quantum épuisé, reste 2u)
- $t=6$  à  $t=8$  : P1 s'exécute (quantum épuisé, reste 1u)
- $t=8$  à  $t=9$  : P2 s'exécute (termine !)
- $t=9$  à  $t=11$  : P3 s'exécute (termine !)
- $t=11$  à  $t=12$  : P1 s'exécute (termine !)

☑ **Résultat : tous les processus progressent régulièrement. Aucun n'est bloqué longtemps.**



## Comparaison FIFO vs Round-Robin

| Critère      | FIFO                                               | Round-Robin                                |
|--------------|----------------------------------------------------|--------------------------------------------|
| Principe     | Premier arrivé = Premier servi                     | Chaque processus reçoit un quantum         |
| Équité       | Non équitable (un processus peut tout monopoliser) | Équitable (tous progressent régulièrement) |
| Réactivité   | Faible (attentes longues)                          | Bonne (alternance rapide)                  |
| Utilisé pour | Traitements par lots (non interactifs)             | Systèmes interactifs (PC, smartphones)     |
| Risque       | Famine des processus courts                        | Overhead si quantum trop petit             |

## 5. Gestion des ressources et exclusion mutuelle

### Qu'est-ce qu'une ressource ?

#### Définition :

Une ressource est tout élément dont un processus a besoin pour s'exécuter.

#### Types de ressources :

- Matérielles : processeur, mémoire RAM, imprimante, carte réseau
- Logicielles : fichier, base de données, variable partagée

### Le problème des ressources partagées

Certaines ressources peuvent être utilisées par plusieurs processus. Cela pose un problème :

#### Exemple concret : deux processus veulent imprimer en même temps

##### P1 (Word) :

- Envoie « Bonjour » à l'imprimante
- ... (interrompu par l'OS)
- Envoie la suite « monde »

##### P2 (PDF) :

- Envoie « 1234 » à l'imprimante
- ... (interrompu par l'OS)
- Envoie la suite « 5678 »

**✗ Résultat sans protection : l'imprimante mélange les deux → « Bon1234jour5678monde »**

### La section critique et l'exclusion mutuelle

| Notion           | Définition                                                     | Exemple                                            |
|------------------|----------------------------------------------------------------|----------------------------------------------------|
| Section critique | Partie du code où le processus accède à une ressource partagée | Les lignes qui envoient des données à l'imprimante |

|                    |                                                                                    |                                       |
|--------------------|------------------------------------------------------------------------------------|---------------------------------------|
| Exclusion mutuelle | Mécanisme qui garantit qu'un seul processus est dans la section critique à la fois | L'OS bloque P2 pendant que P1 imprime |
|--------------------|------------------------------------------------------------------------------------|---------------------------------------|

### **Analogie simple : la salle de bain**

- La salle de bain = la ressource partagée
- Se doucher = la section critique
- La porte verrouillée = l'exclusion mutuelle
- Attendre devant la porte = processus bloqué
- La porte s'ouvre = ressource libérée, processus passe en PRÊT

### **Lien avec les états :**

Quand un processus demande une ressource déjà occupée → il passe à l'état BLOQUÉ.  
Quand la ressource est libérée → le processus passe de BLOQUÉ à PRÊT.

## 6. L'interblocage (Deadlock)

### Définition :

Un interblocage (deadlock) est une situation où deux processus (ou plus) sont bloqués indéfiniment, chacun attendant une ressource détenue par l'autre.

### Exemple concret (à maîtriser pour le Bac !)

Situation : P1 utilise l'imprimante, P2 utilise le scanner

|   | P1 (traitement de texte)                                                                                                                                                                 | P2 (logiciel de scan)                                                                                                                                                                      |
|---|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | Obtient l'imprimante   | Obtient le scanner   |
| 2 | Demande le scanner  → BLOQUÉ !                                                                        | Demande l'imprimante  → BLOQUÉ !                                                                      |
| 3 | Attend P2 libère le scanner...                                                                                                                                                           | Attend P1 libère l'imprimante...                                                                                                                                                           |
| 4 | <b>✗ Ne peut pas avancer</b>                                                                                                                                                             | <b>✗ Ne peut pas avancer</b>                                                                                                                                                               |

 **Résultat : les deux processus sont bloqués pour toujours ! C'est un interblocage.**

### La condition de l'interblocage

Un interblocage se produit quand il y a une attente circulaire :

P1 → attend une ressource de P2

P2 → attend une ressource de P1



 **Cycle = Blocage total**

### Analogie : le carrefour bloqué

➤ 4 voitures arrivent en même temps à un carrefour

- Chacune veut avancer mais est bloquée par celle d'en face
- Personne ne peut bouger → interblocage !
- Solution : une voiture recule (= l'OS interrompt un processus)

### **Comment l'OS peut-il gérer un interblocage ?**

- Détecter l'interblocage et interrompre un des processus
- Prévenir l'interblocage en imposant un ordre d'acquisition des ressources
- Redémarrer le système (cas extrême)

## 7. Synthèse générale du chapitre

| Notion             | Définition simple                                    | Mot-clé             |
|--------------------|------------------------------------------------------|---------------------|
| Processus          | Programme en cours d'exécution                       | dynamique           |
| État prêt          | Attend son tour pour le CPU                          | file d'attente      |
| État en exécution  | Utilise le CPU en ce moment                          | travail actif       |
| État bloqué        | Attend une ressource (pas le CPU)                    | ressource manquante |
| Ordonnanceur       | Chef qui décide quel processus utilise le CPU        | décisionnaire       |
| FIFO               | Premier arrivé = premier servi                       | ordre d'arrivée     |
| Round-Robin        | Chaque processus reçoit un quantum de temps          | équité              |
| Ressource partagée | Utilisée par plusieurs processus                     | concurrence         |
| Exclusion mutuelle | Un seul processus à la fois dans la section critique | verrou              |
| Section critique   | Code qui accède à une ressource partagée             | zone protégée       |
| Interblocage       | Attente circulaire → blocage total                   | deadlock            |



### Questions type Bac — avec réponses détaillées

#### Q1 Quelle est la différence entre un programme et un processus ?

##### Réponse :

Un programme est une suite d'instructions stockées sur le disque (fichier statique). Un processus est ce programme au moment où il s'exécute en mémoire (actif, dynamique). Un même programme peut donner naissance à plusieurs processus.



**Astuce :** Pense à la recette (programme) et au plat en cours de préparation (processus).

#### Q2 Pourquoi un système d'exploitation est-il nécessaire ?

 **Réponse :**

Le système d'exploitation permet de gérer l'exécution de plusieurs processus en partageant le processeur et les ressources. Sans lui, un seul programme pourrait s'exécuter à la fois et il n'y aurait pas de partage des ressources.

 **Astuce :** L'OS est le chef d'orchestre de l'ordinateur.

**Q3 Un processus attend l'accès à une imprimante. Dans quel état est-il ?**

 **Réponse :**

Le processus est dans l'état BLOQUÉ. Il attend une ressource (l'imprimante) qui est actuellement occupée. Il ne consomme pas le CPU pendant cette attente.

 **Astuce :** Dès qu'un processus attend une ressource  $\neq$  CPU  $\rightarrow$  état BLOQUÉ.

**Q4 Quel est le rôle de l'ordonnanceur ?**

 **Réponse :**

L'ordonnanceur est un composant du système d'exploitation qui décide quel processus parmi ceux en état PRÊT va utiliser le processeur. Il organise l'alternance entre les processus selon un algorithme (FIFO, Round-Robin...).

 **Astuce :** L'ordonnanceur ne gère que les processus PRÊTS.

**Q5 Qu'est-ce que l'exclusion mutuelle et pourquoi est-elle nécessaire ?**

 **Réponse :**

L'exclusion mutuelle garantit qu'un seul processus peut accéder à une ressource partagée à la fois. Elle est nécessaire pour éviter les conflits d'accès qui pourraient corrompre les données (ex : deux processus imprimant simultanément = sortie mélangée).

 **Astuce :** Sans exclusion mutuelle = données corrompues.

**Q6 Décrire une situation d'interblocage avec deux processus P1 et P2.**

 **Réponse :**

P1 possède la ressource R1 et demande R2. P2 possède R2 et demande R1. P1 attend que P2 libère R2, et P2 attend que P1 libère R1. Aucun des deux ne peut avancer : c'est un interblocage.

 **Astuce :** Chercher toujours l'attente circulaire pour identifier un deadlock.



## Fiche mémo — À mémoriser absolument

### Les 3 états d'un processus

- PRÊT : attend le CPU
- EN EXÉCUTION : utilise le CPU
- BLOQUÉ : attend une ressource

### Transitions :

- Prêt → Exécution : l'ordonnanceur choisit
- Exécution → Prêt : quantum épuisé
- Exécution → Bloqué : attente ressource
- Bloqué → Prêt : ressource disponible

### Formules à retenir

- Processus = programme EN COURS d'exécution
- Ordonnanceur = décide QUI utilise le CPU
- FIFO = premier arrivé, premier servi
- Round-Robin = quantum de temps pour chacun
- Exclusion mutuelle = UN SEUL à la fois
- Section critique = accès à ressource partagée
- Interblocage = attente CIRCULAIRE entre processus

### ★ Les 4 points ESSENTIELS pour le Bac :

- 1 Un processus est un programme EN COURS d'exécution (pas juste un programme stocké).
- 2 Les 3 états : PRÊT (attend CPU) / EXÉCUTION (utilise CPU) / BLOQUÉ (attend ressource).
- 3 L'ordonnanceur décide quel processus utilise le CPU parmi ceux en état PRÊT.
- 4 L'interblocage = attente circulaire : P1 attend P2, P2 attend P1 → blocage total.